BINF 6210/8210:
Machine Learning for Bioinformatics

Introduction to Neural Networks:
Multilayer Perceptron

Xiuxia Du
Department of Bioinformatics and Genomics
University of North Carolina at Charlotte

Outline

Introduction

Real and Artificial Neuron

Artificial Neural Network

What is Multilayer Perceptron (MLP)?

MLP Learning

Parameter Initialization

Parameter Update Strategies

Introduction

Motivation

Neural Networks in Bioinformatics

- High-dimensional biological data
- nonlinear biological mechanisms
- predictive modeling challenges

Examples of Bioinformatics Problems

- Cancer classification from gene expression
- Metabolomics disease prediction
- Protein function prediction
- Microbiome community analysis

Key message: Biology is rarely linear.

Why Does Nonlinearith Matter?

Biological systems are inherently nonlinear.

- Example A - Gene regulation threshold effect
  - Gene A has no effect until expression exceeds a threshold
  - After threshold $\rightarrow$ strong activation of disease pathway

  This cannot be modeled well by linear regression.
- Example B — Interaction between metabolites. Disease risk may increase only when:
  - Glucose is high AND
  - triglycerides are high

  This is an interaction surface, not a straight line.
- Example C — Saturation effects. Biological receptors often follow:
  - saturation kinetics
  - sigmoid-like response.

  This is naturally nonlinear.

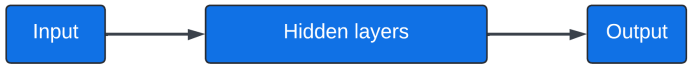
Challenges for Simple Linear Models

Simple linear models may struggle to capture complex biological relationships, such as:

- Interactions among genes, proteins, and metabolites
- Threshold and saturation effects
- Nonlinear regulation of biological pathways

Neural Networks can learn nonlinear relationships and interactions directly from data.

Neural Networks as a Nonlinear Function Approximator



A Neural Network learns a function that maps inputs to outputs:

$$\hat{y} = f_\theta(x)$$

where

- x : Biological measurements (gene expression, metabolites, SNPs, microbiome abundance etc)
- $\hat{y}$: Predicted biological outcome (disease status, survival probability, phenotype, etc)
- θ: Learnable parameters (weights and biases)

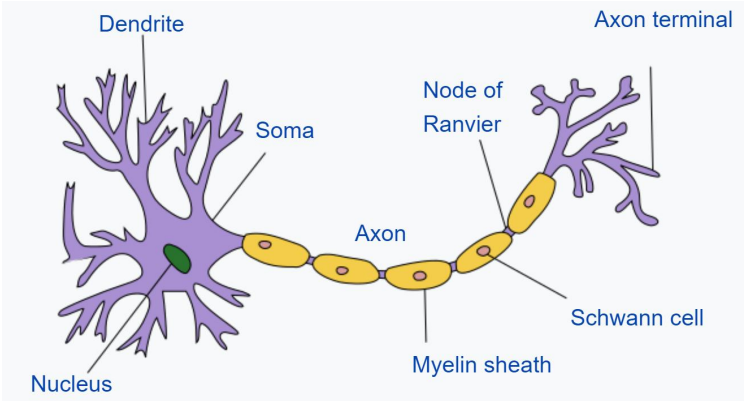
By combining linear transformations with nonlinear activation functions, neural networks can approximate complex relationships in biological data.

Multilayer Perceptron

- Neural networks include many different architectures.
- In this lecture, we focus on **multilayer perceptrons (MLPs)**, a class of feed-forward neural networks consisting of fully connected layers.
- MLPs provide the foundation for understanding more advanced architectures such as CNNs, RNNs, and Transformers.

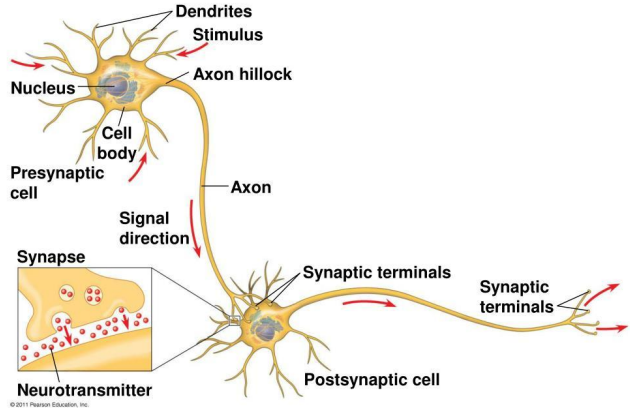
Real and Artificial Neuron

A Real Neuron

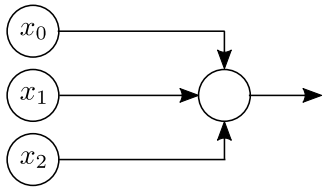


Communications between Real Neurons

Figure 48.4



Artificial Neuron: Mathematical framework



$$h_\theta(x) = \frac{1}{1 + e^{-(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)}}$$

with $x_0 = 1$ representing the bias term and h_θ is the sigmoid/logistic activation function. $\theta_0, \theta_1, \theta_2$ are called weights or parameters. In general:

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix}, \quad h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Activation Function: sigmoid

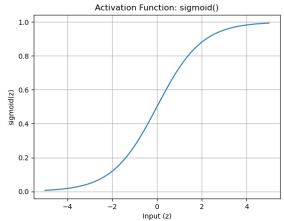
- Linear combination of x:

$$z = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- Sigmoid activation

$$h_\theta(x) = \frac{1}{1 + e^{-z}}$$

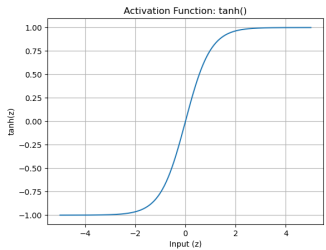
Biological intuition: Sigmoid behaves like biological saturation.



Activation Function: tanh()

The hyperbolic tangent activation function is:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



Properties:

- Output range:

$$-1 < \tanh(z) < 1$$

- Symmetric around zero
- Smooth and differentiable
- For large positive input → output approaches +1
- For large negative input → output approaches -1
- Near zero → approximately linear
- Suitable for gradient-based learning.

Activation Function: tanh()

Intuition: tanh() behaves like a soft biological switch with inhibitory (negative) and activating (positive) states.

Activation Function: tanh()

Derivative is elegant:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$$

Properties:

- derivative largest near $z = 0$
- derivative very small at extremes

This leads to: **Vanishing gradient problem**, i.e. when neuron saturates near ± 1:

- gradient ≈ 0
- learning slows

This is why modern deep learning prefers ReLU.

tanh() and sigmoid()

Relationship between tanh() and sigmoid():

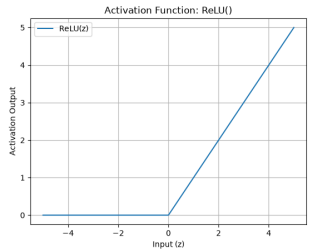
$$\tanh(z) = 2 \cdot \text{sigmoid}(2z) - 1$$

- sigmoid output range $\rightarrow$ 0 to 1
- tanh output range $\rightarrow$ -1 to 1

Activation Function: ReLU()

The Rectified Linear Unit (ReLU) activation function is:

$$\text{ReLU}(z) = \max(0, z)$$



This means:

- If $z < 0 \rightarrow$ output = 0
- If $z \geq 0 \rightarrow$ output = z

So the function is piecewise linear.

Intuition:

- ReLU behaves like a biological activation threshold.
- The neuron is inactive when signal is weak.
- The neuron is linearly responsive when signal exceeds threshold.

Activation Function: ReLU()

Historically:

- sigmoid and tanh were widely used.
- deep networks trained very slowly.

Why ReLU became dominant in deep learning? ReLU improved training because:

- It avoids the vanishing gradient problem for positive z

$$\frac{d}{dz}\text{ReLU}(z) = \begin{cases} 0 & \text{if} \quad z < 0 \\ 1 & \text{if} \quad z > 0 \end{cases}$$

  - Gradients do not shrink in positive z region.
  - Deep networks can learn more efficiently.
  - At $z = 0$, the derivative is mathematically undefined. However, implementations commonly assign it a value of zero.
- Computational simplicity: no exponentials and fast

Activation Function: ReLU()

Why ReLU became dominant in deep learning? ReLU improved training because:

- Sparse activation property
  - many neurons output zero.
  - network becomes sparse.

  Interpretation: Neural network selects only a subset of biological signals to represent disease state.
- Biological interpretation:
  - below threshold → pathway inactive
  - above threshold → activity increases proportionally

  This is realistic for enzyme activation, gene regulation, and signaling cascades. Thus ReLU models **pathway "turn-on" dynamics**.

Activation Function: ReLU()

- Important limitation – Dying ReLU problem

  If neuron receives strongly negative inputs, then it may output 0 for all future inputs. Then gradient = 0 and neuron never recovers. This is called **dying ReLU**.

Activation Function: ReLU()

Modern variants of ReLU:

- Leaky ReLU

$$\text{LeakyReLU}(z) = \begin{cases} 0.01z & \text{for} \quad z < 0 \\ z & \text{for} \quad z \geq 0 \end{cases}$$

- Exponential Linear Unit (ELU)

$$\text{ELU}(z) = \begin{cases} z & \text{for} \quad z \geq 0 \\ \alpha(e^z - 1) & \text{for} \quad z < 0 \end{cases}$$

  ELU behaves like a soft biological suppression response rather than hard shut-off.

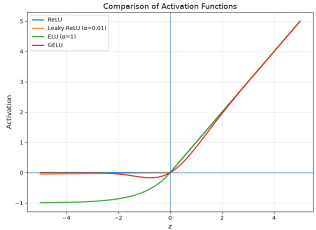
- Gaussian Error Linear Unit (GELU)

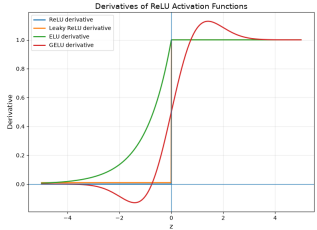
$$\text{GELU}(z) = z\Phi(z)$$

  where $\Phi(z)$ is the cumulative distribution function of the standard normal distribution. GELU is used in transformers, large language models, and modern deep learning architectures.

Activation Function: ReLU()

- **ReLU:** derivative is 0 for all $z \leq 0$, which explains how gradients can vanish completely for inactive neurons.
- **Leaky ReLU:** keeps a small nonzero gradient on the negative side, so neurons can still recover.
- **ELU:** smooth derivative on the negative side, approaching 0 gradually.
- **GELU:** smooth, non-piecewise behavior, with a derivative that changes continuously.





Activation Function: ReLU()

Comparison of ReLU and its variants.

Function	Negative Region	Smoothness	Modern Usage
ReLU	zero flat	non-smooth	very common
ELU	exponential	smooth	moderate
GELU	probabilistic smooth	very smooth	very modern deep learning

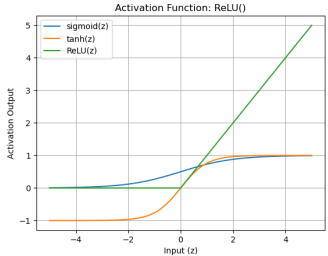
Summary:

- Sigmoid: probabilistic intuition
- tanh: optimization intuition
- ReLU: modern deep learning
- ELU: improvement idea
- GELU: frontier concept

sigmoid, tanh, and ReLU

Function	Output Range	Zero-centered	Training Speed	Modern Use
Sigmoid	0 to 1	No	slow	output layer mainly
tanh	-1 to 1	Yes	moderate	hidden layers
ReLU	0 to ∞	No	fast	dominant in deep learning

tanh() is mathematically elegant even if ReLU is more common in modern architectures.



Artificial Neural Network

What is an Artificial Neural Network (ANN)?

- An artificial neural network is a computational model consisting of interconnected units (neurons) that transform input data into outputs using learnable parameters.
- Neural networks include many architectures:
  - Multilayer perceptrons (MLPs)
  - Convolutional neural networks (CNNs)
  - Recurrent neural networks (RNNs)
  - Transformers
  - Graph neural networks (GNNs)

  These architectures differ in how neurons are connected, how information flows, and how inputs are processed.

ANN Architectures

Architecture	Key characteristic	Bioinformatics application
MLP	Fully connected feed-forward layers	Disease prediction from gene expression
CNN	Convolutional filters exploit local patterns	DNA sequence motif recognition
RNN	Recurrent connections process sequential information	Biological time-series modeling
Transformer	Attention mechanisms model relationships among tokens	Protein language models
GNN	Message passing over graph structures	Molecular property prediction

What is a Deep Neural Network (DNN)?

- A DNN is simply a neural network with multiple hidden layers. It is not a separate architectural family.
- An MLP with one hidden layer is a relatively shallow neural network.
- An MLP with ten hidden layers is both an MLP and a deep neural network.
- A Transformer with many layers is also a deep neural network, but it is not ordinarily described as an MLP as a whole.

Therefore, MLP describes the architecture, whereas deep describes its depth.

Activation Function Usage

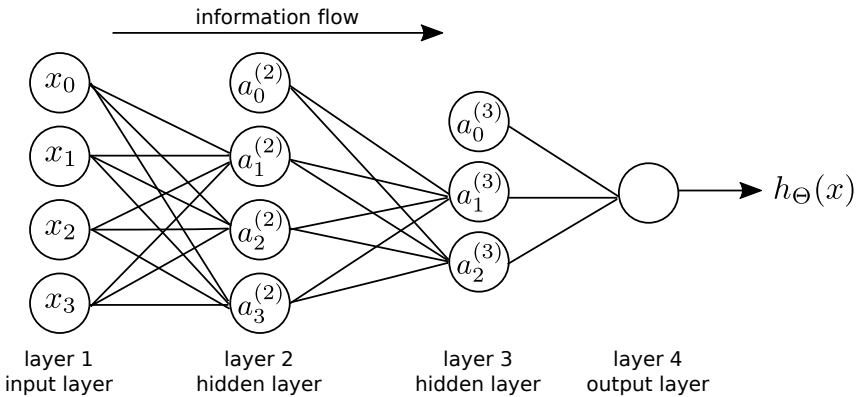
Activation	Typical Applications	Comments
ReLU	MLPs, CNNs	Classic default; simple and effective
Leaky ReLU	MLPs, CNNs	Addresses dying ReLU
GELU	Transformers, language models	Smooth activation; widely used in Transformer feed-forward networks
SiLU (Swish)	Modern CNNs, some language models	Smooth activation; used in many newer architectures
Sigmoid	Binary classification output	Less common in hidden layers

What is Multilayer Perceptron?

What is an MLP?

- An MLP is a feed-forward neural network with one or more hidden layers, typically composed of fully connected (dense) layers.
- An MLP has three defining characteristics:
  - **Feed-forward:** Information moves from input to output without recurrent connections.
  - **Fully connected layers:** Each neuron receives inputs from all neurons in the preceding layer.
  - **Nonlinear activations:** Hidden layers typically apply functions such as ReLU, enabling nonlinear modeling.

General Architecture



Data Transformation

- A neural network does not just fit a function directly.
- It transforms the input data step-by-step into new representations that make prediction easier.

$$x \to h_1 \to h_2 \cdots \to h_\Theta(x)$$

where

- x = original biological measurements
- h_k = hidden layer representation
- $h_\Theta(x)$ = predicted biological outcome

Automatic Feature Learning

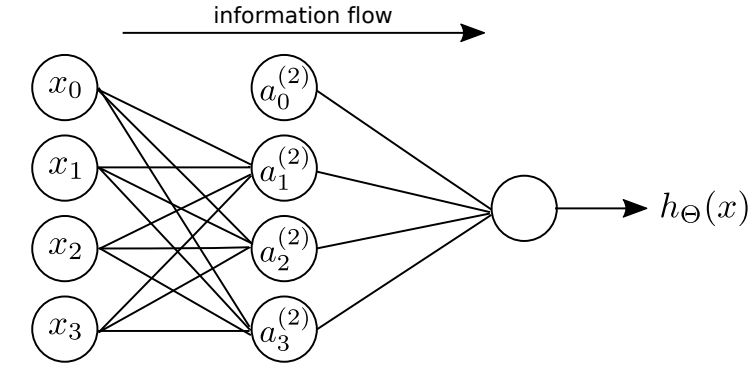
- Neural networks automatically learn useful transformations of input features through hidden layers.
- Unlike traditional machine learning approaches that may require manually engineered features, neural networks jointly optimize feature representations and prediction during training.
- The learned features can capture complex nonlinear relationships, but they are not necessarily biologically interpretable.

From One Neuron to Neural Network

- Logistic regression is a neural network with a single sigmoid output unit and no hidden layers.
- When hidden layers are introduced:
  - The model can learn nonlinear transformations of the input features.
  - Multiple neurons can learn different intermediate representations.
  - The network can model relationships that ordinary logistic regression cannot capture without feature engineering.

MLP Learning

An MLP Example Architecture for Explanation



Each neuron is called a unit.

Forward Propagation

- Denote a matrix of weights, Θ, with the Ith row, $\Theta^{(j)}_{I,:}$, representing the function mapping to unit I in layer $j+1$ from units in layer j.
- Denote $a_i^{(j)}$ to represent the "activation" of unit i in layer j.
- Then we have

$$\begin{aligned} a_1^{(2)} &= g\left(\Theta_{10}^{(1)}x_0 + \Theta_{11}^{(1)}x_1 + \Theta_{12}^{(1)}x_2 + \Theta_{13}^{(1)}x_3\right) \\ a_2^{(2)} &= g\left(\Theta_{20}^{(1)}x_0 + \Theta_{21}^{(1)}x_1 + \Theta_{22}^{(1)}x_2 + \Theta_{23}^{(1)}x_3\right) \\ a_3^{(2)} &= g\left(\Theta_{30}^{(1)}x_0 + \Theta_{31}^{(1)}x_1 + \Theta_{32}^{(1)}x_2 + \Theta_{33}^{(1)}x_3\right) \end{aligned}$$

with $\Theta^{(1)} \in \mathbb{R}^{3\times 4}$ and

$$h_\Theta(x) = a_1^{(3)} = g\left(\Theta_{10}^{(2)}a_0^{(2)} + \Theta_{11}^{(2)}a_1^{(2)} + \Theta_{12}^{(2)}a_2^{(2)} + \Theta_{13}^{(2)}a_3^{(2)}\right)$$

with $\Theta^{(2)} \in \mathbb{R}^{1\times 4}$.

Forward Propagation

- In general, if a network has s_j units in layer j and $s_{(j+1)}$ units in layer $j+1$, then $\Theta^{(j)}$ will be of dimension $s_{(j+1)} \times (s_j + 1)$.

Forward Propagation

- Denote $z_1^{(2)}, z_2^{(2)}, z_3^{(2)}$ as the input to the first, second, and third unit in the hidden layer:

$$\begin{aligned} z_1^{(2)} &= \Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3 \\ z_2^{(2)} &= \Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3 \\ z_3^{(2)} &= \Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3 \end{aligned}$$

  and denote

$$z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

- Then we have

$$\begin{aligned} z^{(2)} &= \Theta^{(1)} x \\ a^{(2)} &= g\left(z^{(2)}\right) \end{aligned}$$

  where $g(\cdot)$ applies to each element of $z^{(2)}$ separately.

Forward Propagation

- Define

$$a^{(1)} := x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

then

$$\begin{aligned} z^{(2)} &= \Theta^{(1)} a^{(1)} \\ a^{(2)} &= g\left(z^{(2)}\right) \end{aligned}$$

with $a^{(1)} \in \mathbb{R}^{4\times 1}$, $\Theta^{(1)} \in \mathbb{R}^{3\times 4}$, $z^{(2)} \in \mathbb{R}^{3\times 1}$ and $a^{(2)} \in \mathbb{R}^{3\times 1}$.

- Add $a_0^{(2)} = 1$, then we have

$$\begin{aligned} z^{(3)} &= \Theta^{(2)} a^{(2)} \\ h_\Theta(x) &= a^{(3)} = g\left(z^{(3)}\right) \end{aligned}$$

Forward Propagation

- In general,

$$\begin{aligned} z^{(j)} &= \Theta^{(j-1)} a^{(j-1)} \\ a^{(j)} &= g\left(z^{(j)}\right) \end{aligned}$$

- If the network has a total of n layers, then

$$h_\Theta(x) = a^{(n)} = g\left(z^{(n)}\right)$$

- Remark: The forward propagation in a neural network consists of successive affine transformations and activation functions.

Forward Propagation

When the bias term is explicit, the general forward propagation is described as:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$
$$a^{(l)} = g^{(l)}(z^{(l)})$$

Cost Function for Regression

Mean Squared Error (MSE)

- Used when the neural network predicts continuous outcomes.
- For m training samples and K output units:

$$J_{\text{MSE}}(\Theta) = \frac{1}{2m} \sum_{i=1}^{m} \sum_{k=1}^{K} \left(\hat{y}_k^{(i)} - y_k^{(i)} \right)^2$$

- $y_k^{(i)}$: observed value for output k of sample i.
- $\hat{y}_k^{(i)}$: corresponding neural network prediction.
- $K = 1$ for a single continuous outcome.
- $K > 1$ for multiple continuous outcomes.

Bioinformatics example: Predicting metabolite concentrations from gene expression.

Cost Function for Binary Classification

Binary Cross-Entropy (BCE)

- Used for binary classification with a sigmoid output.
- For each sample:

$$y^{(i)} \in \{0, 1\}, \qquad \hat{y}^{(i)} \in (0, 1)$$

- The cost function is:

$$J_{\text{BCE}}(\Theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

- Penalizes confident incorrect predictions.
- Produces a low loss when predicted probabilities agree with the true labels.

Bioinformatics example: Predicting whether a patient has a particular disease.

Cost Function for Multiclass Classification

Categorical Cross-Entropy (CCE)

- Used for single-label multiclass classification with a softmax output layer.
- For K mutually exclusive classes:

$$y_k^{(i)} \in \{0, 1\}, \qquad \sum_{k=1}^{K} y_k^{(i)} = 1$$

- Predicted class probabilities satisfy:

$$\sum_{k=1}^{K} \hat{y}_k^{(i)} = 1$$

$$J_{\text{CCE}}(\Theta) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log \hat{y}_k^{(i)}$$

Bioinformatics example: Classifying tumor samples into one of several cancer subtypes.

Overall Training Cycle

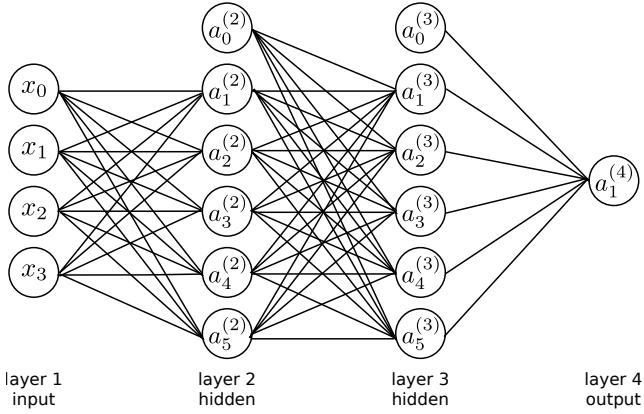
- Step 1: Forward Propagation
  Input data moves through network:
  $$x \to h_1 \to h_2 \to \cdots \to h_\Theta(x)$$
  and produces prediction.
- Step 2: Compute Cost
- Step 3: Back Propagation
  We compute:
  $$\frac{\partial J}{\partial \theta}$$
  for every θ in the network. This tells us which weights contributed to the cost and by how much.
- Step 4: Parameter Update
  Gradient descent update:
  $$\theta \leftarrow \theta - \alpha \frac{\partial J}{\partial \theta}$$
  Learning improves prediction.

Neural Network for Classification

Notations:

- Samples: $\{\left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \cdots, \left(x^{(m)}, y^{(m)}\right)\}$
- L: total number of layers in the network
- s_l: Number of units (not counting bias unit) in layer l
- K: number of classes

Binary Classification

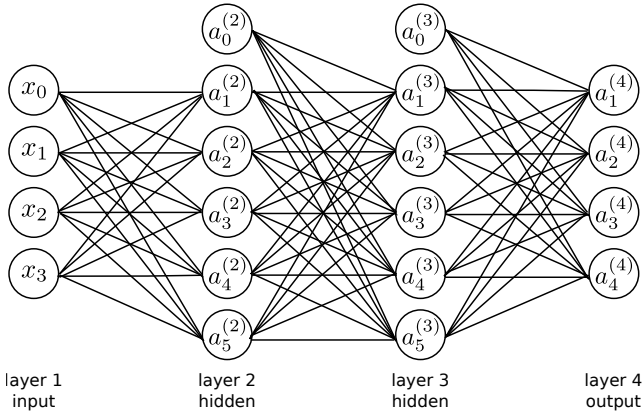


$$y \in \{0, 1\}$$

$$K = 1, h_\Theta(x) \in \mathbb{R}^K, \text{i.e. } h_\Theta(x) \in \mathbb{R}$$

$$L = 4, s_1 = 3, s_2 = 5, s_3 = 5, s_4 = 1$$

Multiclass Classification



Multiclass Classification

- This is for $K \geq 3$ classification.
- For the example on the previous slide, $K = 4$, $y \in \mathbb{R}^K$ and we use

$$y = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\}$$

to represent 4 classes, e.g., pedestrian, car, motorcycle, and truck, respectively.
$L = 4, s_1 = 3, s_2 = 5, s_3 = 5, s_4 = 4$
$h_\Theta(x) \in \mathbb{R}^4$

Cost Function for Regression

- Use the mean squared error
- Linear regression

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2 = \frac{1}{2m} \sum_{i=1}^{m} \left(\theta^T x_i - y_i \right)^2$$

- Neural network

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \sum_{k=1}^{K} \left[\left(h_\theta(x^{(i)}) \right)_k - y_k^{(i)} \right]^2$$

where K is the total number of output units.

Minimize the Cost Function

- Next we need to find Θ that would minimize $J(\Theta)$.
- The minimization requires code to compute
  - $J(\Theta)$
  - $\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$
- Computation of $J(\Theta)$ is straightforward.
- The optimal Θ can be found by gradient descent.

Gradient Descent Analogy

Imagine:

- Cost function = mountain landscape
- parameter values determine your current location
- High cost $\rightarrow$ high altitude and low cost $\rightarrow$ low altitude

Training process: gradient descent is like walking downhill trying to reach the lowest valley.

Gradient

- Gradient is defined as:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \\ \frac{\partial J}{\partial \theta_d} \end{bmatrix}$$

Gradient Descent

- Gradient tells us which direction increases J fastest.
- So negative gradient tells us the direction of steepest decrease in J.
- Gradient descent tells us how to update the θ values based on the current values as follows:

$$\theta_{\text{new}} := \theta_{\text{old}} - \alpha \nabla J(\theta)$$

where α, a positive real number, is the learning rate.

Training Cycle

- Neural Network makes prediction using forward propagation.
- Compute cost.
- Compute gradient using back propagation.
- Update parameters Θ.

Repeat this process thousands of times.

Gradient Descent Example

- Cost function:
$$J(\theta) = (\theta - 3)^2$$
- Find θ where J is the minimum.
- Gradient:
$$\frac{dJ}{d\theta} = 2(\theta - 3)$$
- θ update rule:
$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot 2(\theta - 3)$$
- We can see that:
  - $\theta < 3 \rightarrow$ gradient is negative $\rightarrow \theta_{\text{new}}$ moves right
  - $\theta > 3 \rightarrow$ gradient is positive $\rightarrow \theta_{\text{new}}$ moves left

Learning Rate Intuition

Learning rate α intuition

- Learning rate controls the step size when walking downhill.
- Too small $\rightarrow$ very slow training
- Too large $\rightarrow$ overshoot valley and unstable training
- Good learning rate $\rightarrow$ smooth convergence.

Analogy: like adjusting walking stride when descending a mountain.

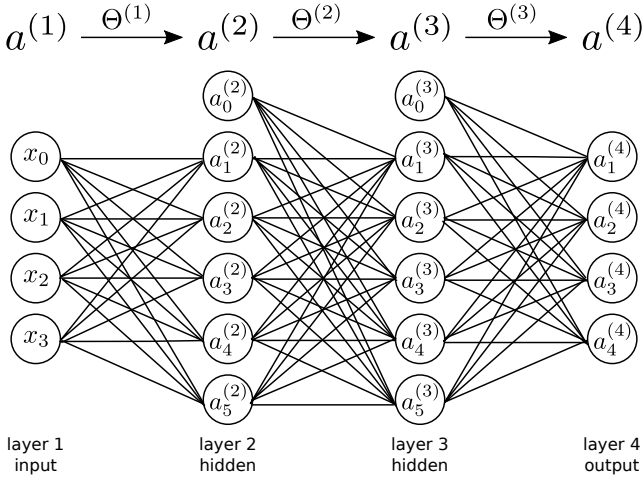
High-Dimensional Reality in Neural Network

- A real neural network could have millions of parameters and the cost function surface is extremely complex.
- Gradient descent guides the navigation in a high-dimensional error landscape to reach the minimum.
- Training is an iterative improvement and there is no instant solution.

Local vs Global Minimum

- Neural network creates a cost function surface that is:
  - extremely high-dimensional
  - highly non-convex
  - full of complex curvature
- This cost function surface is characterized by:
  - many valleys
  - flat plateaus
  - saddle points
- So the optimization becomes a very complex navigation problem.
- Usually the final minimum reached during neural network training is a local minimum (or saddle-region), not guaranteed to be the global minimum.
- In deep learning, many local minima have similar cost values and some local minima generalize better. Therefore, reaching global minimum is not always necessary.

Summary: Forward Propagation



Summary: Forward Propagation

- Given one training sample (x, y), forward propagation is done as follows for the neural network architecture shown on the previous slide:

$$
\begin{aligned}
a^{(1)} &= x \\
z^{(2)} &= \Theta^{(1)} a^{(1)} \\
a^{(2)} &= g\left(z^{(2)}\right) \\
&\text{then add } a_0^{(2)} \text{ to } a^{(2)} \\
z^{(3)} &= \Theta^{(2)} a^{(2)} \\
a^{(3)} &= g\left(z^{(3)}\right) \\
&\text{then add } a_0^{(3)} \text{ to } a^{(3)} \\
z^{(4)} &= \Theta^{(3)} a^{(3)} \\
a^{(4)} &= g\left(z^{(4)}\right) = h_\Theta(x)
\end{aligned}
$$

Backpropagation

- Neural networks contain:
  - many layers
  - millions of parameters
- We cannot adjust weights by intuition. Backpropapation provides **systematic credit assignment mechanism**
- Backpropagation relies on the **chain rule of calculus**

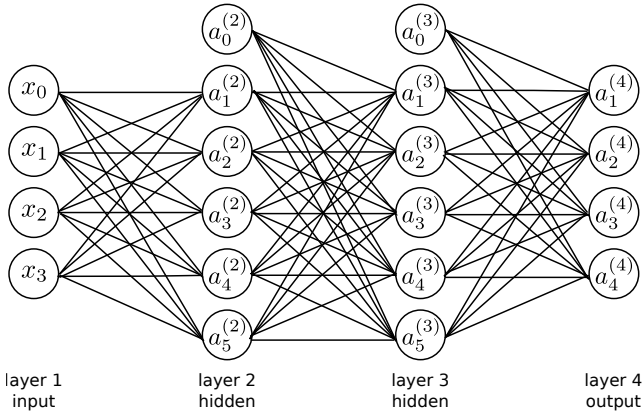
$$\theta \rightarrow z \rightarrow h_\Theta \rightarrow J$$

Then:

$$\frac{\partial J}{\partial \theta} = \frac{\partial J}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial \theta}$$

Backpropagation

$$\delta^{(2)} \xleftarrow[a^{(2)}]{\Theta^{(2)}} \delta^{(3)} \xleftarrow[a^{(3)}]{\Theta^{(3)}} \delta^{(4)}$$



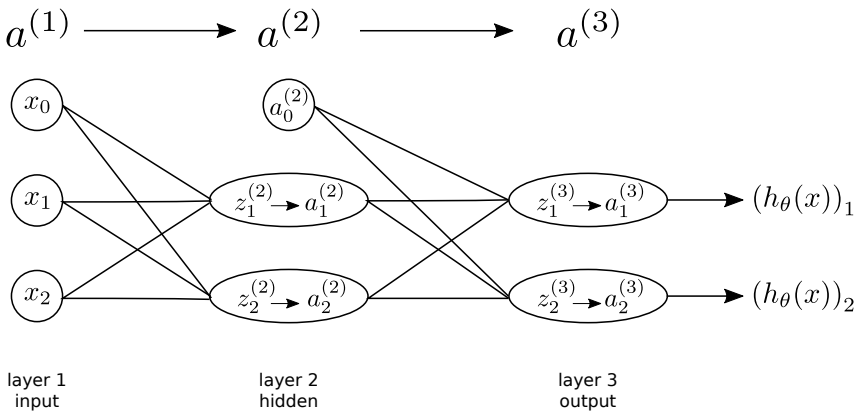
Backpropagation

- Use backpropagation to calculate

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$$

for all i, j and $l = 1, 2, \cdots, L-1$.

Example Architecture for Explaining Backpropagation



Example: Forward Propagation

- From $a^{(1)}$ to $a^{(2)}$:

$$z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \end{bmatrix} = \begin{bmatrix} \Theta_{10}^{(1)} & \Theta_{11}^{(1)} & \Theta_{12}^{(1)} \\ \Theta_{20}^{(1)} & \Theta_{21}^{(1)} & \Theta_{22}^{(1)} \end{bmatrix} \begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ a_2^{(1)} \end{bmatrix}, \quad a^{(2)} = \begin{bmatrix} a_1^{(2)} \\ a_2^{(2)} \end{bmatrix} = g\left(z^{(2)}\right)$$

- From $a^{(2)}$ to $a^{(3)}$:

$$z^{(3)} = \begin{bmatrix} z_1^{(3)} \\ z_2^{(3)} \end{bmatrix} = \begin{bmatrix} \Theta_{10}^{(2)} & \Theta_{11}^{(2)} & \Theta_{12}^{(2)} \\ \Theta_{20}^{(2)} & \Theta_{21}^{(2)} & \Theta_{22}^{(2)} \end{bmatrix} \begin{bmatrix} a_0^{(2)} \\ a_1^{(2)} \\ a_2^{(2)} \end{bmatrix}, \quad a^{(3)} = \begin{bmatrix} a_1^{(3)} \\ a_2^{(3)} \end{bmatrix} = g\left(z^{(3)}\right)$$

- Total cost:

$$J(\Theta) = \frac{1}{2}\left[\left(y_1 - a_1^{(3)}\right)^2 + \left(y_2 - a_2^{(3)}\right)^2\right]$$

Example: Backpropagation

- The goal is to calculate $\frac{\partial J\Theta}{\partial \Theta_{ij}^{(l)}}$ for $l = 1, 2$. We will first compute these partial derivatives for $l = 2$ and then for $l = 1$.
- $l = 2$:

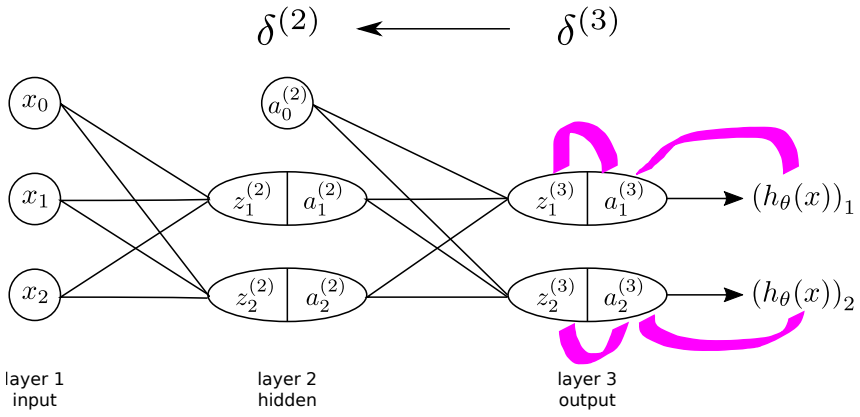
$$\left[\frac{\partial J(\Theta)}{\partial \Theta_{10}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{11}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{12}^{(2)}}\right] = \frac{\partial J(\Theta)}{\partial z_1^{(3)}} \cdot \left[a_0^{(2)}, a_1^{(2)}, a_2^{(2)}\right]$$

and

$$\left[\frac{\partial J(\Theta)}{\partial \Theta_{20}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{21}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{22}^{(2)}}\right] = \frac{\partial J(\Theta)}{\partial z_2^{(3)}} \cdot \left[a_0^{(2)}, a_1^{(2)}, a_2^{(2)}\right]$$

where $\frac{\partial J(\Theta)}{\partial z_1^{(3)}} \in \mathbb{R}$ and $\frac{\partial J(\Theta)}{\partial z_2^{(3)}} \in \mathbb{R}$.

Example: Backpropagation



Example: Backpropagation

- By the chain rule in calculus, we have

$$\frac{\partial J(\Theta)}{\partial z_1^{(3)}} = \frac{\partial J(\Theta)}{\partial a_1^{(3)}} \cdot \frac{\partial a_1^{(3)}}{\partial z_1^{(3)}} = \left(a_1^{(3)} - y_1\right) \cdot \left[a_1^{(3)} \cdot \left(1 - a_1^{(3)}\right)\right]$$

  with

$$\frac{\partial J(\Theta)}{\partial a_1^{(3)}} = a_1^{(3)} - y_1$$

  for the mean squared error cost function and

$$\frac{\partial a_1^{(3)}}{\partial z_1^{(3)}} = a_1^{(3)} \cdot \left(1 - a_1^{(3)}\right)$$

  for the logistic activation function.

- Similarly,

$$\frac{\partial J(\Theta)}{\partial z_2^{(3)}} = \frac{\partial J(\Theta)}{\partial a_2^{(3)}} \cdot \frac{\partial a_2^{(3)}}{\partial z_2^{(3)}} = \left(a_2^{(3)} - y_2\right) \cdot \left[a_2^{(3)} \cdot \left(1 - a_2^{(3)}\right)\right]$$

Example: Backpropagation

- Define

$$\delta^{(3)} := \begin{bmatrix} \frac{\partial J(\Theta)}{\partial z_1^{(3)}} \\ \frac{\partial J(\Theta)}{\partial z_2^{(3)}} \end{bmatrix} = \begin{bmatrix} \left(a_1^{(3)} - y_1\right) \cdot \left[a_1^{(3)} \cdot \left(1 - a_1^{(3)}\right)\right] \\ \left(a_2^{(3)} - y_2\right) \cdot \left[a_2^{(3)} \cdot \left(1 - a_2^{(3)}\right)\right] \end{bmatrix}$$

- Then

$$\begin{bmatrix} \frac{\partial J(\Theta)}{\partial \Theta_{10}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{11}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{12}^{(2)}} \\ \frac{\partial J(\Theta)}{\partial \Theta_{20}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{21}^{(2)}}, \frac{\partial J(\Theta)}{\partial \Theta_{22}^{(2)}} \end{bmatrix} = \delta^{(3)} \cdot \left[a_0^{(2)}, a_1^{(2)}, a_2^{(2)}\right]$$

Example: Backpropagation

- Calculate $\frac{\partial J\Theta}{\partial \Theta_{ij}^{(l)}}$ for $l = 1$:

$$\left[\frac{\partial J(\Theta)}{\partial \Theta_{10}^{(1)}}, \frac{\partial J(\Theta)}{\partial \Theta_{11}^{(1)}}, \frac{\partial J(\Theta)}{\partial \Theta_{12}^{(1)}}\right] = \frac{\partial J(\Theta)}{\partial z_1^{(2)}} \cdot \left[a_0^{(1)}, a_1^{(1)}, a_2^{(1)}\right]$$

and

$$\left[\frac{\partial J(\Theta)}{\partial \Theta_{20}^{(1)}}, \frac{\partial J(\Theta)}{\partial \Theta_{21}^{(1)}}, \frac{\partial J(\Theta)}{\partial \Theta_{22}^{(1)}}\right] = \frac{\partial J(\Theta)}{\partial z_2^{(2)}} \cdot \left[a_0^{(1)}, a_1^{(1)}, a_2^{(1)}\right]$$

where $\frac{\partial J(\Theta)}{\partial z_1^{(2)}} \in \mathbb{R}$ and $\frac{\partial J(\Theta)}{\partial z_2^{(2)}} \in \mathbb{R}$.

Example: Backpropagation

- The total cost is the sum of the costs for unit 1 and unit 2 in the output layer:

$$J(\Theta) = \frac{1}{2m} \sum_{i=1}^{m} \sum_{k=1}^{K} \left[\left(h_\Theta(x^{(i)}) \right)_k - y_k^{(i)} \right]^2$$

- For a single sample $\{(x, y)\}$, the total cost can be written as

$$J(\Theta) = \frac{1}{2} \left[\left(a_1^{(3)} - y_1 \right)^2 + \left(a_2^{(3)} - y_2 \right)^2 \right]$$

- Define $J_1(\Theta)$ and $J_2(\Theta)$ as the cost for unit 1 and unit 2 in the output layer, respectively, i.e.

$$J_1(\Theta) = \frac{1}{2} \left(a_1^{(3)} - y_1 \right)^2, \quad J_2(\Theta) = \frac{1}{2} \left(a_2^{(3)} - y_2 \right)^2$$

- Then

$$J(\Theta) = J_1(\Theta) + J_2(\Theta)$$

Example: Backpropagation

- Now we compute $\frac{\partial J(\Theta)}{\partial z_1^{(2)}}$.

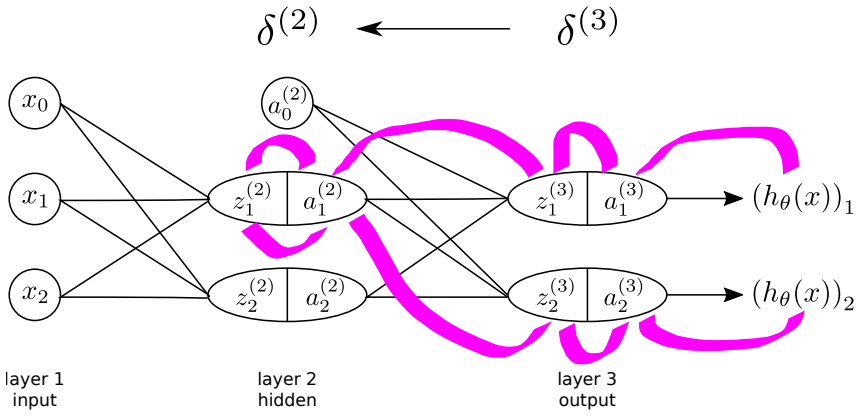
$$\frac{\partial J(\Theta)}{\partial z_1^{(2)}} = \frac{\partial J_1(\Theta)}{\partial z_1^{(2)}} + \frac{\partial J_2(\Theta)}{\partial z_1^{(2)}}$$

with

$$\begin{aligned}
\frac{\partial J_1(\Theta)}{\partial z_1^{(2)}} &= \frac{\partial J_1(\Theta)}{\partial a_1^{(3)}} \cdot \frac{\partial a_1^{(3)}}{\partial z_1^{(3)}} \cdot \frac{\partial z_1^{(3)}}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\
&= \left(a_1^{(3)} - y_1\right) \cdot \left[a_1^{(3)}\left(1 - a_1^{(3)}\right)\right] \cdot \Theta_{11}^{(2)} \cdot \left[a_1^{(2)}\left(1 - a_1^{(2)}\right)\right]
\end{aligned}$$

$$\begin{aligned}
\frac{\partial J_2(\Theta)}{\partial z_1^{(2)}} &= \frac{\partial J_2(\Theta)}{\partial a_2^{(3)}} \cdot \frac{\partial a_2^{(3)}}{\partial z_2^{(3)}} \cdot \frac{\partial z_2^{(3)}}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\
&= \left(a_2^{(3)} - y_2\right) \cdot \left[a_2^{(3)}\left(1 - a_2^{(3)}\right)\right] \cdot \Theta_{21}^{(2)} \cdot \left[a_1^{(2)}\left(1 - a_1^{(2)}\right)\right]
\end{aligned}$$

Example: Backpropagation



Example: Backpropagation

- Similarly for $\frac{\partial J(\Theta)}{\partial z_2^{(2)}}$, we have

$$\frac{\partial J(\Theta)}{\partial z_2^{(2)}} = \frac{\partial J_1(\Theta)}{\partial z_2^{(2)}} + \frac{\partial J_2(\Theta)}{\partial z_2^{(2)}}$$

with

$$\begin{aligned}
\frac{\partial J_1(\Theta)}{\partial z_2^{(2)}} &= \frac{\partial J_1(\Theta)}{\partial a_1^{(3)}} \cdot \frac{\partial a_1^{(3)}}{\partial z_1^{(3)}} \cdot \frac{\partial z_1^{(3)}}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \\
&= \left(a_1^{(3)} - y_1\right) \cdot \left[a_1^{(3)}\left(1 - a_1^{(3)}\right)\right] \cdot \Theta_{12}^{(2)} \cdot \left[a_2^{(2)}\left(1 - a_2^{(2)}\right)\right]
\end{aligned}$$

$$\begin{aligned}
\frac{\partial J_2(\Theta)}{\partial z_2^{(2)}} &= \frac{\partial J_2(\Theta)}{\partial a_2^{(3)}} \cdot \frac{\partial a_2^{(3)}}{\partial z_2^{(3)}} \cdot \frac{\partial z_2^{(3)}}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \\
&= \left(a_2^{(3)} - y_2\right) \cdot \left[a_2^{(3)}\left(1 - a_2^{(3)}\right)\right] \cdot \Theta_{22}^{(2)} \cdot \left[a_2^{(2)}\left(1 - a_2^{(2)}\right)\right]
\end{aligned}$$

Example: Backpropagation

- Define

$$\delta^{(2)} := \begin{bmatrix} \dfrac{\partial J(\Theta)}{\partial z_1^{(2)}} \\ \dfrac{\partial J(\Theta)}{\partial z_2^{(2)}} \end{bmatrix}$$

- Then

$$\begin{bmatrix} \dfrac{\partial J(\Theta)}{\partial \Theta_{10}^{(1)}}, \dfrac{\partial J(\Theta)}{\partial \Theta_{11}^{(1)}}, \dfrac{\partial J(\Theta)}{\partial \Theta_{12}^{(1)}} \\ \dfrac{\partial J(\Theta)}{\partial \Theta_{20}^{(1)}}, \dfrac{\partial J(\Theta)}{\partial \Theta_{21}^{(1)}}, \dfrac{\partial J(\Theta)}{\partial \Theta_{22}^{(1)}} \end{bmatrix} = \delta^{(2)} \cdot \left[a_0^{(1)}, a_1^{(1)}, a_2^{(1)} \right]$$

- Recall that $a_0^{(1)} = a_0^{(2)} = 1$.
- This completes one iteration of the forward and backpropagation.
- Continue this process until $\Theta_{ij}^{(l)}$ converge for $l = 1, 2$.

Example: Backpropagation

- How do we streamline the process of back propagation from $\delta^{(j+1)}$ to $\delta^{(j)}$?

Example: Backpropagation

Note that:

$$\left[\frac{\partial J(\Theta)}{\partial a_1^{(2)}}, \frac{\partial J(\Theta)}{\partial a_2^{(2)}}\right] = \left[\left(a_1^{(3)} - y_1\right) \cdot \left[a_1^{(3)}\left(1 - a_1^{(3)}\right)\right], \left(a_2^{(3)} - y_2\right) \cdot \left[a_2^{(3)}\left(1 - a_2^{(3)}\right)\right]\right] \begin{bmatrix} \Theta_{11}^{(2)} & \Theta_{12}^{(2)} \\ \Theta_{21}^{(2)} & \Theta_{22}^{(2)} \end{bmatrix}$$

$$= \left[\delta_1^{(3)}, \delta_2^{(3)}\right] \begin{bmatrix} \Theta_{11}^{(2)} & \Theta_{12}^{(2)} \\ \Theta_{21}^{(2)} & \Theta_{22}^{(2)} \end{bmatrix} = \left(\delta^{(3)}\right)^T \begin{bmatrix} \Theta_{11}^{(2)} & \Theta_{12}^{(2)} \\ \Theta_{21}^{(2)} & \Theta_{22}^{(2)} \end{bmatrix}$$

Example: Backpropagation

and

$$\left(\delta^{(2)}\right)^T = \left[\frac{\partial J(\Theta)}{\partial z_1^{(2)}}, \frac{\partial J(\Theta)}{\partial z_2^{(2)}}\right] = \left[\frac{\partial J(\Theta)}{\partial a_1^{(2)}}, \frac{\partial J(\Theta)}{\partial a_2^{(2)}}\right] \begin{bmatrix} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} & 0 \\ 0 & \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \end{bmatrix}$$

$$= \left(\delta^{(3)}\right)^T \begin{bmatrix} \Theta_{11}^{(2)} & \Theta_{12}^{(2)} \\ \Theta_{21}^{(2)} & \Theta_{22}^{(2)} \end{bmatrix} \begin{bmatrix} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} & 0 \\ 0 & \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \end{bmatrix}$$

Example: Backpropagation

- In general,

$$\left(\delta^{(j)}\right)^T = \left(\delta^{(j+1)}\right)^T \Theta^{(j)} \begin{bmatrix} \frac{\partial a_1^{(j)}}{\partial z_1^{(j)}} & 0 & \cdots & 0 \\ 0 & \frac{\partial a_2^{(j)}}{\partial z_2^{(j)}} & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & \frac{\partial a_{s_j}^{(j)}}{\partial z_{s_j}^{(j)}} \end{bmatrix}$$

where $\left(\delta^{(j)}\right)^T$ is of size $1 \times s_j$, $\left(\delta^{(j+1)}\right)^T$ is of size $1 \times s_{(j+1)}$, $\Theta^{(j)}$ is of size $s_{(j+1)} \times s_j$, and the diagonal matrix is of size $s_j \times s_j$.

Example: Backpropagation

- Backpropagation starts with computing $\delta^{(L)}$.
- If neural network is used for regression, the total cost is

$$J(\Theta) = \frac{1}{2m} \sum_{i=1}^{m} \sum_{k=1}^{K} \left[\left(h_\theta(x^{(i)}) \right)_k - y_k^{(i)} \right]^2$$

  where m is the total number of samples and K is the total number of output units.
- For a specific sample, the associated cost is

$$J(\Theta) = \frac{1}{2} \sum_{k=1}^{K} \left[(h_\theta(x))_k - y_k \right]^2 = \frac{1}{2} \sum_{k=1}^{K} \left[a_k^{(L)} - y_k \right]^2$$

Example: Backpropagation

$\delta^{(L)}$ can be computed as

$$
\begin{aligned}
\delta^{(L)} &= \begin{bmatrix} \frac{\partial J(\Theta)}{\partial z_1^{(L)}} \\ \frac{\partial J(\Theta)}{\partial z_2^{(L)}} \\ \vdots \\ \frac{\partial J(\Theta)}{\partial z_{s_L}^{(L)}} \end{bmatrix} = \begin{bmatrix} \frac{\partial J(\Theta)}{\partial a_1^{(L)}} \cdot \frac{\partial a_1^{(L)}}{\partial z_1^{(L)}} \\ \frac{\partial J(\Theta)}{\partial a_2^{(L)}} \cdot \frac{\partial a_2^{(L)}}{\partial z_2^{(L)}} \\ \vdots \\ \frac{\partial J(\Theta)}{\partial a_{s_L}^{(L)}} \cdot \frac{\partial a_{s_L}^{(L)}}{\partial z_{s_L}^{(L)}} \end{bmatrix} = \begin{bmatrix} \frac{\partial J(\Theta)}{\partial a_1^{(L)}} \\ \frac{\partial J(\Theta)}{\partial a_2^{(L)}} \\ \vdots \\ \frac{\partial J(\Theta)}{\partial a_{s_L}^{(L)}} \end{bmatrix} \cdot \begin{bmatrix} \frac{\partial a_1^{(L)}}{\partial z_1^{(L)}} & 0 & \cdots & 0 \\ 0 & \frac{\partial a_2^{(L)}}{\partial z_2^{(L)}} & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & \frac{\partial a_{s_L}^{(L)}}{\partial z_{s_L}^{(L)}} \end{bmatrix} \\
&= \begin{bmatrix} \left(a_1^{(L)} - y_1\right) \\ \left(a_2^{(L)} - y_2\right) \\ \vdots \\ \left(a_{s_L}^{(L)} - y_{s_L}\right) \end{bmatrix} \cdot \begin{bmatrix} a_1^{(L)}\left(1 - a_1^{(L)}\right) & 0 & \cdots & 0 \\ 0 & a_2^{(L)}\left(1 - a_2^{(L)}\right) & \cdots & 0 \\ \vdots & & & \\ 0 & 0 & \cdots & a_{s_L}^{(L)}\left(1 - a_{s_L}^{(L)}\right) \end{bmatrix}
\end{aligned}
$$

Example: Backpropagation

- If a neural network is used for classification, the total cost is

$$J(\Theta) = -\frac{1}{m}\left\{\sum_{I=1}^{m}\sum_{k=1}^{K}\left[y_k^{(i)}\log\left(h_\Theta(x^{(i)})\right)_k + \left(1-y_k^{(i)}\right)\log\left(1-\left(h_\Theta(x^{(i)})\right)_k\right)\right]\right\}$$

  where m is the total number of samples and K is the total number of output units.

- For a specific sample, the associated cost is

$$\begin{aligned} J(\Theta) &= -\sum_{k=1}^{K}\left[y_k\log\left(h_\Theta(x)\right)_k + (1-y_k)\log\left(1-\left(h_\Theta(x)\right)_k\right)\right] \\ &= -\sum_{k=1}^{K}\left[y_k\log a_k^{(L)} + (1-y_k)\log\left(1-a_k^{(L)}\right)\right] \end{aligned}$$

Example: Backpropagation

$\delta^{(L)}$ can be computed as

$$
\begin{aligned}
\delta^{(L)} &= \begin{bmatrix} \frac{\partial J(\Theta)}{\partial z_1^{(L)}} \\ \frac{\partial J(\Theta)}{\partial z_2^{(L)}} \\ \vdots \\ \frac{\partial J(\Theta)}{\partial z_{s_L}^{(L)}} \end{bmatrix} = \begin{bmatrix} \frac{\partial J(\Theta)}{\partial a_1^{(L)}} \cdot \frac{\partial a_1^{(L)})}{\partial z_1^{(L)}} \\ \frac{\partial J(\Theta)}{\partial a_2^{(L)}} \cdot \frac{\partial a_2^{(L)})}{\partial z_2^{(L)}} \\ \vdots \\ \frac{\partial J(\Theta)}{\partial a_{s_L}^{(L)}} \cdot \frac{\partial a_{s_L}^{(L)})}{\partial z_{s_L}^{(L)}} \end{bmatrix} = \begin{bmatrix} \frac{\partial J(\Theta)}{\partial a_1^{(L)}} \\ \frac{\partial J(\Theta)}{\partial a_2^{(L)}} \\ \vdots \\ \frac{\partial J(\Theta)}{\partial a_{s_L}^{(L)}} \end{bmatrix} \cdot \begin{bmatrix} \frac{\partial a_1^{(L)}}{\partial z_1^{(L)}} & 0 & \cdots & 0 \\ 0 & \frac{\partial a_2^{(L)}}{\partial z_2^{(L)}} & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & \frac{\partial a_{s_L}^{(L)}}{\partial z_{s_L}^{(L)}} \end{bmatrix} \\
&= \begin{bmatrix} \frac{\left(a_1^{(L)} - y_1\right)}{a_1^{(L)}\left(1 - a_1^{(L)}\right)} \\ \frac{\left(a_2^{(L)} - y_2\right)}{a_2^{(L)}\left(1 - a_2^{(L)}\right)} \\ \vdots \\ \frac{\left(a_{s_L}^{(L)} - y_{s_L}\right)}{a_{s_L}^{(L)}\left(1 - a_{s_L}^{(L)}\right)} \end{bmatrix} \cdot \begin{bmatrix} a_1^{(L)}\left(1 - a_1^{(L)}\right) & 0 & \cdots & 0 \\ 0 & a_2^{(L)}\left(1 - a_2^{(L)}\right) & \cdots & 0 \\ \vdots & & & \\ 0 & 0 & \cdots & a_{s_L}^{(L)}\left(1 - a_{s_L}^{(L)}\right) \end{bmatrix} = \begin{bmatrix} a_1^{(L)} - y_1 \\ a_2^{(L)} - y_2 \\ \vdots \\ a_{s_L}^{(L)} - y_{s_L} \end{bmatrix}
\end{aligned}
$$

Parameter Initialization

Parameter Initialization

- Before training, the learnable parameters (weights and biases) must be initialized.
- **Why not initialize all weights to zero?**
  - Neurons in the same hidden layer would produce identical outputs.
  - Their gradients would remain identical.
  - They would learn the same features.
  - This is known as the **symmetry problem**.
- **Solution:**
  - Initialize weights randomly to break symmetry.
  - Bias parameters ($\Theta_{i0}^{(l)}$) can generally be initialized to zero.
- **The scale of initial weights matters:**
  - Weights that are too small may cause signals and gradients to shrink across layers.
  - Weights that are too large may cause exploding signals or saturated activations.
- **Common initialization methods:**
  - Xavier/Glorot initialization: often used with tanh.
  - He/Kaiming initialization: often used with ReLU.

Xavier and He Weight Initialization

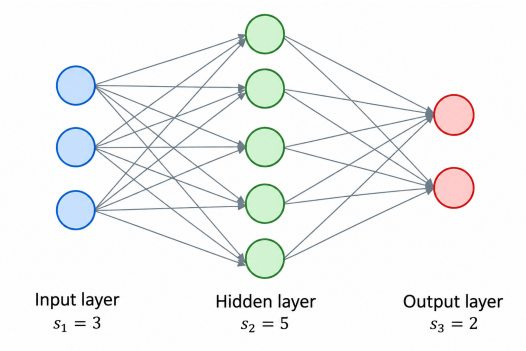
- **Goal:** Maintain reasonable magnitudes of activations and gradients across layers.
- Two commonly used initialization methods:

Method	Weight distribution	Common activation
Xavier/Glorot	$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$	Tanh
He/Kaiming	$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$	ReLU

where, for a particular fully connected layer $l + 1$:

- $n_{\text{in}} = s_l$: number of neurons in the preceding layer l (fan-in).
- $n_{\text{out}} = s_{l+1}$: number of neurons in the current layer $l + 1$ (fan-out).
- Bias units are excluded from both counts.
- $\mathcal{N}(\mu, \sigma^2)$ denotes a normal distribution with mean μ and variance σ^2.

Example: Fan-In and Fan-Out



Fan-in and fan-out:

Layer	n_{in}	n_{out}
Hidden (2)	3	5
Output (3)	5	2

- **Hidden layer:**

$$n_{\text{in}} = s_1 = 3, \quad n_{\text{out}} = s_2 = 5$$

- **Output layer:**

$$n_{\text{in}} = s_2 = 5, \quad n_{\text{out}} = s_3 = 2$$

Key insight: Initialization variance depends on layer dimensions: Xavier uses fan-in and fan-out, while He uses fan-in.

Example: Layer-Specific Weight Initialization

Xavier initialization

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

$$\mathrm{Var}(W^{(1)}) = \frac{2}{3+5} = 0.25$$

$$\mathrm{Var}(W^{(2)}) = \frac{2}{5+2} \approx 0.286$$

He initialization

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}}}$$

$$\mathrm{Var}(W^{(1)}) = \frac{2}{3} \approx 0.667$$

$$\mathrm{Var}(W^{(2)}) = \frac{2}{5} = 0.40$$

Key insight: Each layer uses its own fan-in and fan-out to determine the initialization variance.

Mathematical Foundation of Weight Initialization: Assumptions

Consider the pre-activation of a neuron:

$$z = \sum_{j=1}^{n_{\text{in}}} w_j x_j$$

where n_{in} is the number of incoming connections.

Assumptions:

- Weights w_j are independent, with

$$\mathbb{E}[w_j] = 0, \qquad \text{Var}(w_j) = \sigma_w^2.$$

- Weights are independent of the inputs x_j.
- Inputs have a common second moment:

$$\mathbb{E}[x_j^2] = q_x.$$

Mathematical Foundation of Weight Initialization: Derivation

Recall the pre-activation:

$$z = \sum_{j=1}^{n_{\text{in}}} w_j x_j$$

Variance derivation:

$$\begin{aligned}
\text{Var}(z) &= \sum_{j=1}^{n_{\text{in}}} \text{Var}(w_j x_j) \\
&= \sum_{j=1}^{n_{\text{in}}} \sigma_w^2 \mathbb{E}[x_j^2] \\
&= n_{\text{in}} \sigma_w^2 q_x
\end{aligned}$$

Key insights:

- Pre-activation variance increases with the number of inputs.
- Larger initial weight variance produces larger pre-activation variance.
- To preserve signal variance, weight variance should scale inversely with fan-in.

If the inputs are zero-centered, $q_x = \text{Var}(x)$, giving

$$\boxed{\text{Var}(z) = n_{\text{in}} \text{Var}(w) \text{Var}(x)}$$

Why Xavier and He Initialization Differ

Xavier/Glorot initialization

- Designed to balance forward and backward signal propagation.
- For approximately linear activations (e.g., tanh near zero):
  - Preserving variance during forward propagation:

$$\mathrm{Var}(w) \approx \frac{1}{n_{\text{in}}}$$

  - Preserving variance during backpropagation:

$$\mathrm{Var}(w) \approx \frac{1}{n_{\text{out}}}$$

- Xavier initialization balances these requirements:

$$\boxed{\mathrm{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}}$$

Why Xavier and He Initialization Differ

Key insight: Forward propagation and backpropagation impose different variance-preservation requirements on the same weights. Xavier initialization balances these requirements using both fan-in and fan-out.

Why Xavier and He Initialization Differ

- **He/Kaiming initialization**
  - Designed for ReLU-type activations.
  - For symmetric pre-activations:

$$\mathbb{E}[\mathrm{ReLU}(z)^2] = \frac{1}{2}\mathbb{E}[z^2]$$

  - To compensate for this reduction:

$$\boxed{\mathrm{Var}(w) = \frac{2}{n_{\text{in}}}}$$

Key insight: Initialization accounts for both layer dimensions and activation functions to help stabilize neural network training.

Weight and Bias Initialization Strategies

Strategy	Weights	Biases	Recommendation
A	All zero	All zero	Avoid for hidden layers: symmetry problem
B	Xavier or He	All zero	**Recommended default**
C	Xavier or He	Small random values	Valid, usually unnecessary
D	Xavier or He	Large random values	Generally avoid

Key points:

- Random weight initialization breaks symmetry between neurons in the same hidden layer.
- Biases can be initialized to zero because random weights already break symmetry.
- Random bias initialization is valid, but usually offers little benefit for standard MLPs.

Common practice: Initialize weights using Xavier or He initialization and initialize biases to zero.

Parameter Update Strategies

Parameter Update Strategy: Full-Batch Gradient Descent

- Batch gradient descent uses the **entire training dataset** to compute the gradient before updating model parameters.
- The average loss over N training samples is

$$J(\boldsymbol{\Theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}^{(i)}(\boldsymbol{\Theta})$$

  where
  - $\mathcal{L}^{(i)}(\boldsymbol{\Theta})$: loss for the i-th training sample.
  - $\boldsymbol{\Theta}$: all trainable model parameters (weights and biases).
  - N: number of training samples.
- The parameters are then updated using

$$\boxed{\boldsymbol{\Theta} \leftarrow \boldsymbol{\Theta} - \alpha \nabla_{\boldsymbol{\Theta}} J(\boldsymbol{\Theta})}$$

  where α is the learning rate and $\nabla_{\boldsymbol{\Theta}} J(\boldsymbol{\Theta})$ denotes the gradient of $J(\boldsymbol{\Theta})$.

Parameter Update Strategy: Full-Batch Gradient Descent

- **Characteristics:**
  - Parameters are updated once after processing the entire training dataset.
  - Produces deterministic parameter updates for fixed model parameters and training data.
- **Limitations:**
  - Each update requires processing all training samples.
  - Computationally expensive for very large datasets.

Strategies to Update Parameters: Stochastic GD (SGD)

- Uses **one training sample at a time** to compute the gradient and update parameters.
- For a selected training sample i, the gradient is

$$\nabla_{\Theta} \mathcal{L}^{(i)}(\Theta)$$

- Parameter update:

$$\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} \mathcal{L}^{(i)}(\Theta)$$

- **Characteristics:**
  - Parameters are updated after processing each training sample.
  - Gradient directions can vary substantially between training samples.
  - Updates depend on the order in which training samples are processed.
- **Limitations:**
  - Parameter updates can fluctuate considerably.
  - The training loss may fluctuate rather than decrease smoothly across updates.

Strategies to Update Parameters: Mini-Batch Gradient Descent

- Uses a small subset of training samples (a **mini-batch**) to compute the gradient before updating parameters.
- For a mini-batch $\mathcal{B}$ containing B training samples, the average loss over the batch $\mathcal{B}$ is

$$J_{\mathcal{B}}(\Theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathcal{L}^{(i)}(\Theta)$$

  where $B = |\mathcal{B}|$ and $\mathcal{B}$ is the set of samples in the batch.
- Parameter update:

$$\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} J_{\mathcal{B}}(\Theta)$$

- **Characteristics:**
  - Parameters are updated after processing each mini-batch.
  - Gradient directions are generally less variable than those of SGD.
  - Efficient computation on modern hardware, particularly GPUs.
- **Limitations:**
  - Gradient estimates still vary across mini-batches.
  - Mini-batch size must be selected.

Effect of Batch Size on Optimization

Effect on the Optimization Path

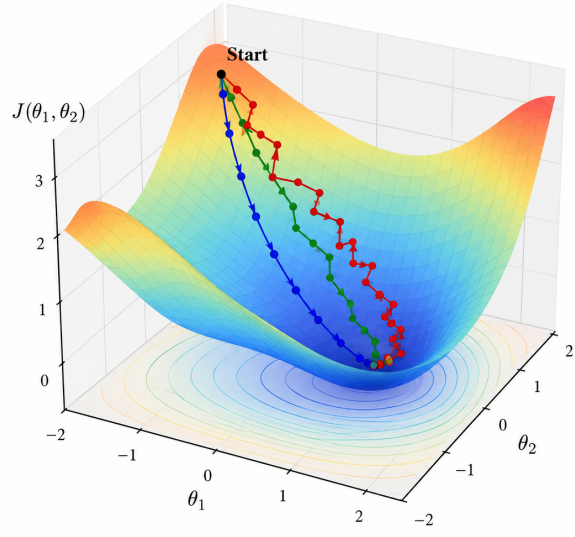
- **Full-Batch Gradient Descent:** Uses the gradient computed from all training samples, producing deterministic parameter updates.
- **Stochastic Gradient Descent (SGD):** Uses one sample per update, resulting in highly variable gradient directions.
- **Mini-Batch Gradient Descent:** Uses the average gradient over a subset of samples, reducing variability compared with SGD.

Effect of Mini-Batch Size

- **Smaller batches:** More variable gradient estimates and more frequent parameter updates.
- **Larger batches:** Generally yield lower-variance estimates of the full training gradient
- **Computational efficiency:** Larger batches can improve parallel computation, but require more memory.

Key Insight: Batch size determines the tradeoff between gradient variability, update frequency, and computational efficiency.

Illustrative optimization trajectories



Full-batch GD
Stochastic GD (SGD)
Mini-batch GD

Full-batch GD

- Uses the exact gradient of the training objective.
- Deterministic updates.
- Smooth trajectory that follows the gradient direction.

Stochastic GD (SGD)

- Uses one sample per update.
- Highly variable gradient directions.
- Noisy trajectory that can explore the loss surface.

Mini-batch GD

- Uses an average gradient over a subset of samples.
- Less noisy than SGD.
- Balances gradient noise and computational efficiency.

Epoch

- An **epoch** is one complete pass through the training dataset.
- During each epoch, the model processes all training samples and updates its parameters.
- The number of parameter updates per epoch depends on the batch size.
- **Illustrative training progression:**
  - **Epoch 1:** The model may still underfit the training data.
  - **Epoch 20:** The model may have learned important patterns.
  - **Epoch 100:** The model may fit the training data more closely.
- **Important:** More epochs do not necessarily improve generalization. Excessive training may lead to overfitting.

Key insight: An epoch measures training progress in terms of data passes, not model accuracy.

Epochs and Parameter Updates

The number of parameter updates per epoch depends on the batch size B and the number of training samples m.

Strategy	Batch Size	Updates per Epoch
Full-Batch Gradient Descent	$B = m$	1
Stochastic Gradient Descent (SGD)	$B = 1$	m
Mini-Batch Gradient Descent	$1 < B < m$	$\lceil m/B \rceil$

Example: For $m = 200$ training samples and $B = 20$, mini-batch gradient descent performs

$$\left\lceil \frac{200}{20} \right\rceil = 10 \text{ updates per epoch.}$$

Note: The formula $\lceil m/B \rceil$ assumes that the final incomplete batch is included.

Computing the Batch Gradient

Define the average loss over a batch $\mathcal{B}$:

$$J_{\mathcal{B}}(\Theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} L^{(i)}(\Theta), \qquad B = |\mathcal{B}|$$

Taking the gradient with respect to $\Theta^{(l)}$:

$$\nabla_{\Theta^{(l)}} J_{\mathcal{B}}(\Theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\Theta^{(l)}} L^{(i)}(\Theta)$$

Mini-Batch Gradient Computation

For layer l, incorporate the bias into the parameter matrix:

$$\boldsymbol{\Theta}^{(l)} = \begin{bmatrix} \mathbf{b}^{(l)} & \mathbf{W}^{(l)} \end{bmatrix}, \qquad \tilde{\mathbf{a}}^{(l,i)} = \begin{bmatrix} 1 \\ \mathbf{a}^{(l,i)} \end{bmatrix}.$$

The augmented activation vector includes a constant 1 for the bias.

The affine transformation becomes

$$\mathbf{z}^{(l+1,i)} = \boldsymbol{\Theta}^{(l)} \tilde{\mathbf{a}}^{(l,i)}.$$

For training sample i, define the backpropagation error signal:

$$\boldsymbol{\delta}^{(l+1,i)} = \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{z}^{(l+1,i)}}.$$

The gradient is given by the outer product:

$$\nabla_{\boldsymbol{\Theta}^{(l)}} \mathcal{L}^{(i)}(\boldsymbol{\Theta}) = \boldsymbol{\delta}^{(l+1,i)} \left(\tilde{\mathbf{a}}^{(l,i)} \right)^T.$$

Averaging Gradients Over a Mini-Batch

Let $\mathcal{B}$ denote a mini-batch of B samples, where $B = |\mathcal{B}|$.
The average mini-batch loss is

$$J_{\mathcal{B}}(\mathbf{\Theta}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathcal{L}^{(i)}(\mathbf{\Theta}).$$

The corresponding gradient for layer l is

$$\nabla_{\mathbf{\Theta}^{(l)}} J_{\mathcal{B}}(\mathbf{\Theta}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \boldsymbol{\delta}^{(l+1,i)} \left(\tilde{\mathbf{a}}^{(l,i)} \right)^T.$$

The parameter update is

$$\boxed{\mathbf{\Theta}^{(l)} \leftarrow \mathbf{\Theta}^{(l)} - \alpha \nabla_{\mathbf{\Theta}^{(l)}} J_{\mathcal{B}}(\mathbf{\Theta})}$$

Important

Compute all sample gradients using the same parameter values, average them, and then update the parameters once per mini-batch.

General Gradient Descent Algorithm

Algorithm:

1. Initialize the parameters $\Theta^{(1)}, \ldots, \Theta^{(L-1)}$.
2. Choose the learning rate α, batch size B, and number of epochs.
3. For each epoch:
   - 3.1 Randomly shuffle the training samples.
   - 3.2 Divide the samples into batches $\mathcal{B}_1, \mathcal{B}_2, \ldots$.
   - 3.3 For each batch $\mathcal{B}_j$:
     - 3.3.1 Compute the average gradients $D^{(l)}$ for all layers using forward propagation and backpropagation.
     - 3.3.2 Update the parameters:

$$\boxed{\Theta^{(l)} \leftarrow \Theta^{(l)} - \alpha D^{(l)}} \quad l = 1, \ldots, L-1$$

Important:

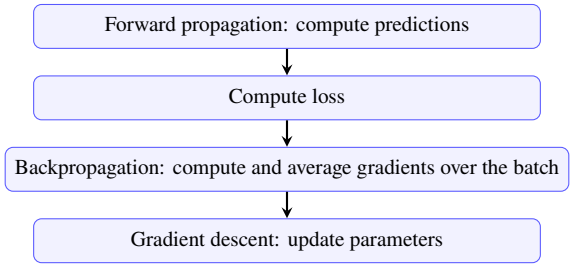
- All gradients within a batch are computed using the same parameter values.
- Parameters are updated only after all layer gradients have been computed.
- The final batch may contain fewer than B samples.

Neural Network Training Workflow

Step 1: Initialize Parameters (Once)

- Initialize weights using Xavier / He initialization.
- Initialize biases to zero.

Step 2: Training Loop (Repeated for Each Batch)



Parameter update:

$$\Theta^{(l)} \leftarrow \Theta^{(l)} - \alpha D^{(l)}$$

- Repeat the training loop over batches and epochs.

Thank you!